

Introduction to Computer Organization

Michael C. Hackett

Associate Professor, Computer Science



Lecture Topics

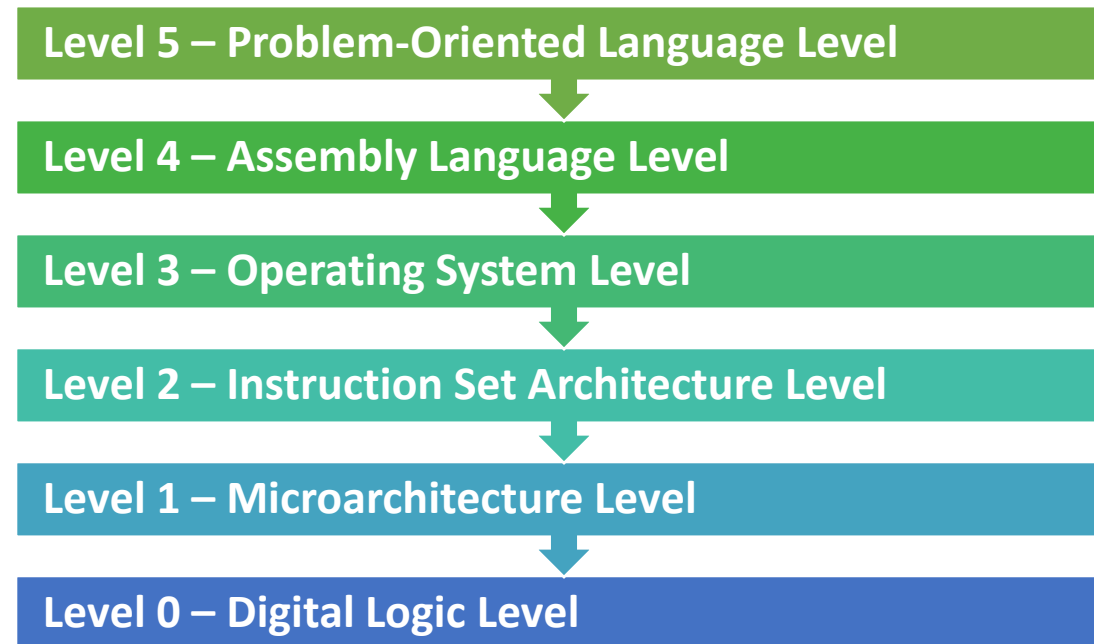
- Multi-Level Machines
- Generations of Computers
 - Mechanical
 - Vacuum Tubes
 - Transistors
 - Integrated Circuits
 - VLSI

Computer Organization

- Computer organization is the systematic design of the interconnecting components that comprise a digital computer.
- It refers to the overall organization and implementation of the computer system.
 - From the lowest level (hardware) to the highest level (software)

Computer Organization

- Modern computer systems are organized as multi-level machines.
 - Instructions from one level are converted to instructions that the next level understands.



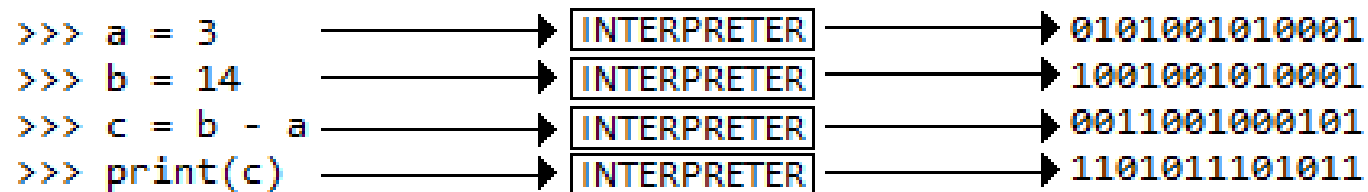
Computer Organization

- Instructions may be converted from one level to the next by translation.
 - The entire program is translated from one level to another by a compiler.



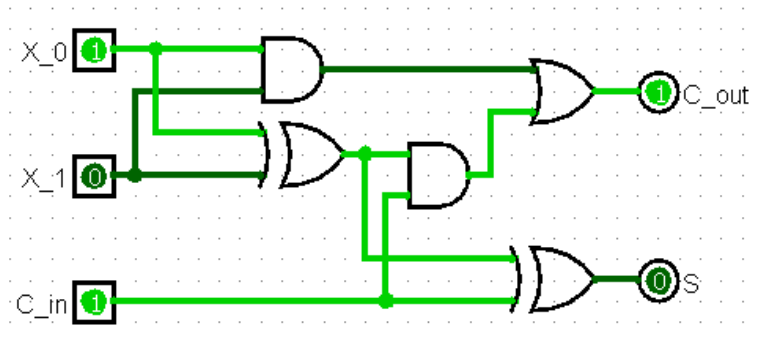
Computer Organization

- Instructions may also be converted from one level to the next by interpretation.
 - Individual instructions are interpreted from one level to another by an interpreter.



Computer Organization

- Logic gates with one or more input signals (0 or 1).
- Gates compute outputs as functions of their inputs.
- Form the hardware components of the microarchitecture.



Level 5 – Problem-Oriented Language Level

Level 4 – Assembly Language Level

Level 3 – Operating System Level

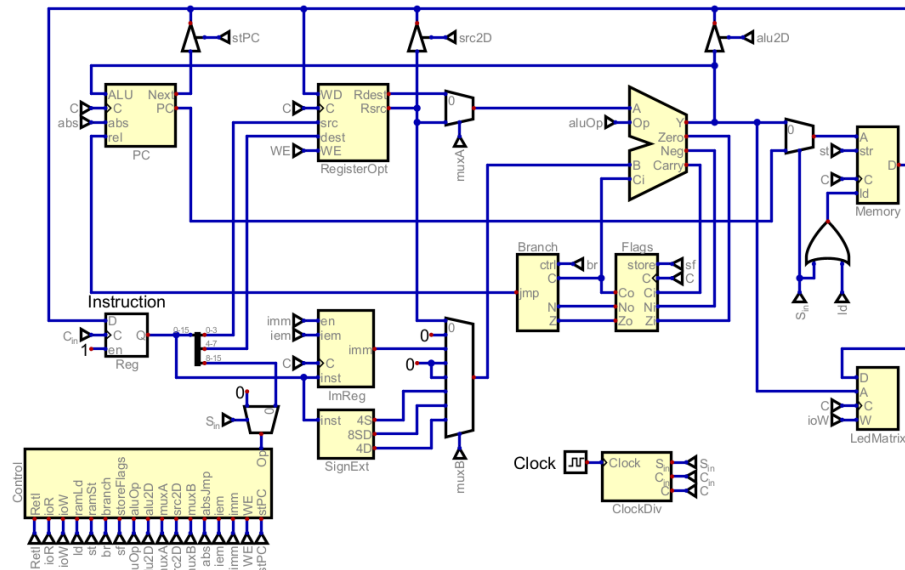
Level 2 – Instruction Set Architecture Level

Level 1 – Microarchitecture Level

Level 0 – Digital Logic Level

Computer Organization

- Interconnections of the physical components in the computer system.
- Forms a datapath for information flow.
- May be controlled by microprograms.



Level 5 – Problem-Oriented Language Level



Level 4 – Assembly Language Level



Level 3 – Operating System Level



Level 2 – Instruction Set Architecture Level



Level 1 – Microarchitecture Level



Hardware

Level 0 – Digital Logic Level

Computer Organization

- The design of the microarchitecture's instructions.
- These instructions can be executed by the microarchitecture level.

Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	0 / 2 _{hex}
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0 (2)	0 _{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0 (2,6)	0 _{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0 (6)	0 / 2 _{hex}
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt	0 / 0 _{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >>> shamt	0 / 0 _{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm(7:0)] = R[rt](7:0) (2)	2 _{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (atomic) ? 1 : 0 (2,7)	3 _{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm(15:0)] = R[rt](15:0) (2)	2 _{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt] (2)	2 _{hex}
Subtract	sub	R	R[rd] = R[rs] - R[rt] (1)	0 / 22 _{hex}
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt] (1)	0 / 23 _{hex}

(1) May cause overflow exception
 (2) SignExtImm = { 16[immediate[15]], immediate }
 (3) ZeroExtImm = { 16[1b'0], immediate }
 (4) BranchAddr = { 14[immediate[15]], immediate, 2'b0 }
 (5) JumpAddr = { PC+4[31:28], address, 2'b0 }
 (6) Operands considered unsigned numbers (vs. 2's comp.)
 (7) Atomic test&set pair; R[rt] = 1 if pair atomic, 0 if not atomic

BASIC INSTRUCTION FORMATS

R	opcode	rs	rt	rd	shamt	funct
31	26-25	21-20	16-15	11-10	6-5	0

I	opcode	rs	rt	immediate
31	26-25	21-20	16-15	0

J	opcode	address
31	26-25	0

Double M[R[rs]+SignExtImm+4] = F[rt+1]

FLOATING-POINT INSTRUCTION FORMATS

FR	opcode	fmat	ft	fs	fd	funct
31	26-25	21-20	16-15	11-10	6-5	0

FI	opcode	fmat	ft	immediate
31	26-25	21-20	16-15	0

PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
Branch Less Than	blt	if(R[rs]<R[rt]) PC = Label
Branch Greater Than	bgt	if(R[rs]>R[rt]) PC = Label
Branch Less Than or Equal	bltle	if(R[rs]<=R[rt]) PC = Label
Branch Greater Than or Equal	bgtle	if(R[rs]>=R[rt]) PC = Label
Load Immediate	li	R[rd] = immediate
Move	move	R[rd] = R[rs]

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

Level 5 – Problem-Oriented Language Level



Level 4 – Assembly Language Level



Level 3 – Operating System Level



Level 2 – Instruction Set Architecture Level

Direct Execution



Interpretation/Microprograms

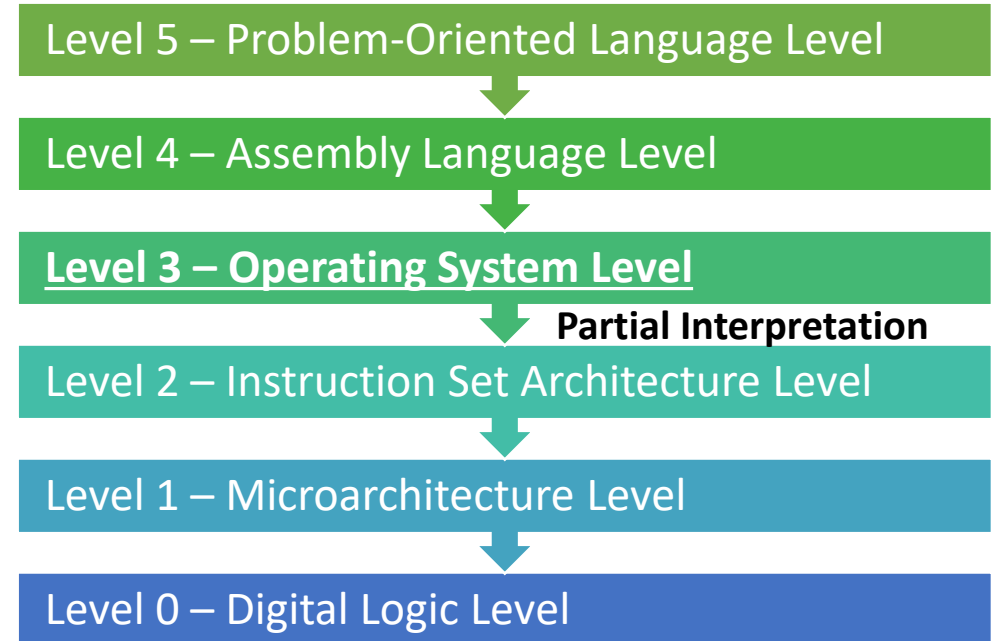
Level 1 – Microarchitecture Level



Level 0 – Digital Logic Level

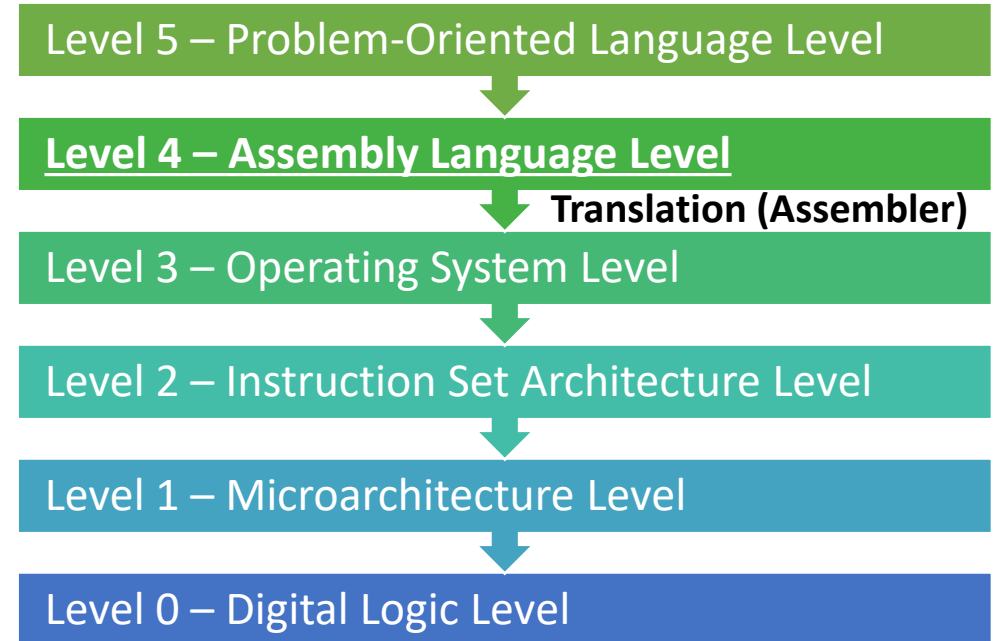
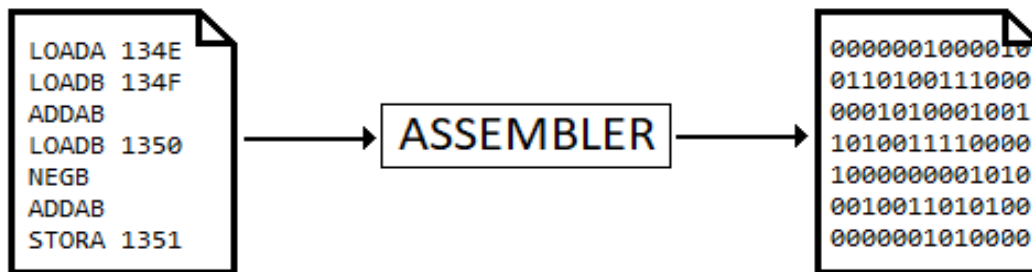
Computer Organization

- Hybrid level between the lower levels and higher levels.
- The operating system can interpret some instructions at this level as Level 2 instructions.
- Some instructions may be executed directly by Level 1 microprograms.



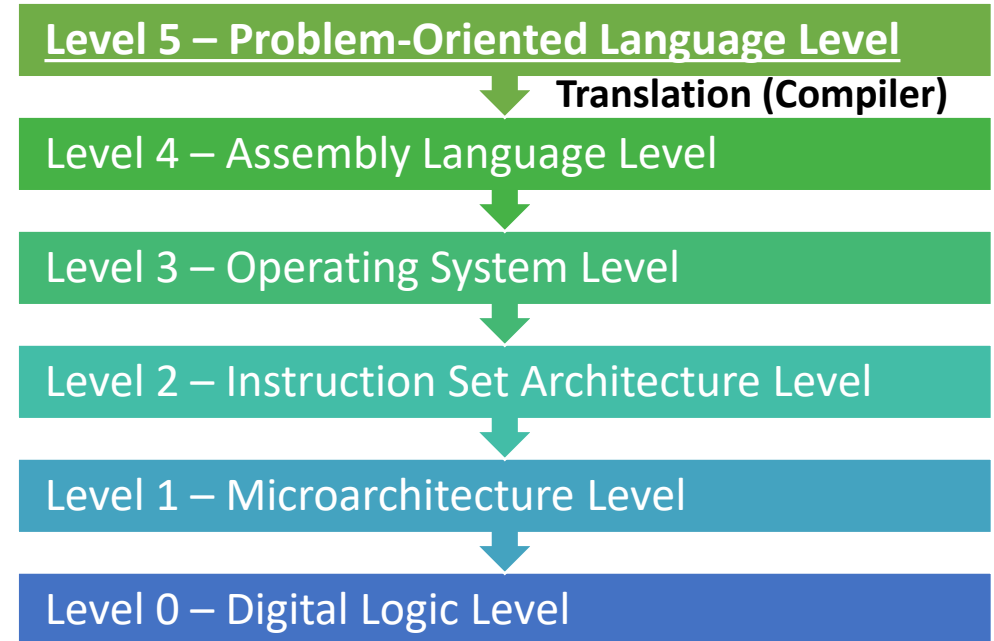
Computer Organization

- Symbolic form of a language of a lower level.
 - Typically, the instruction set.
- Programs are translated using an assembler.



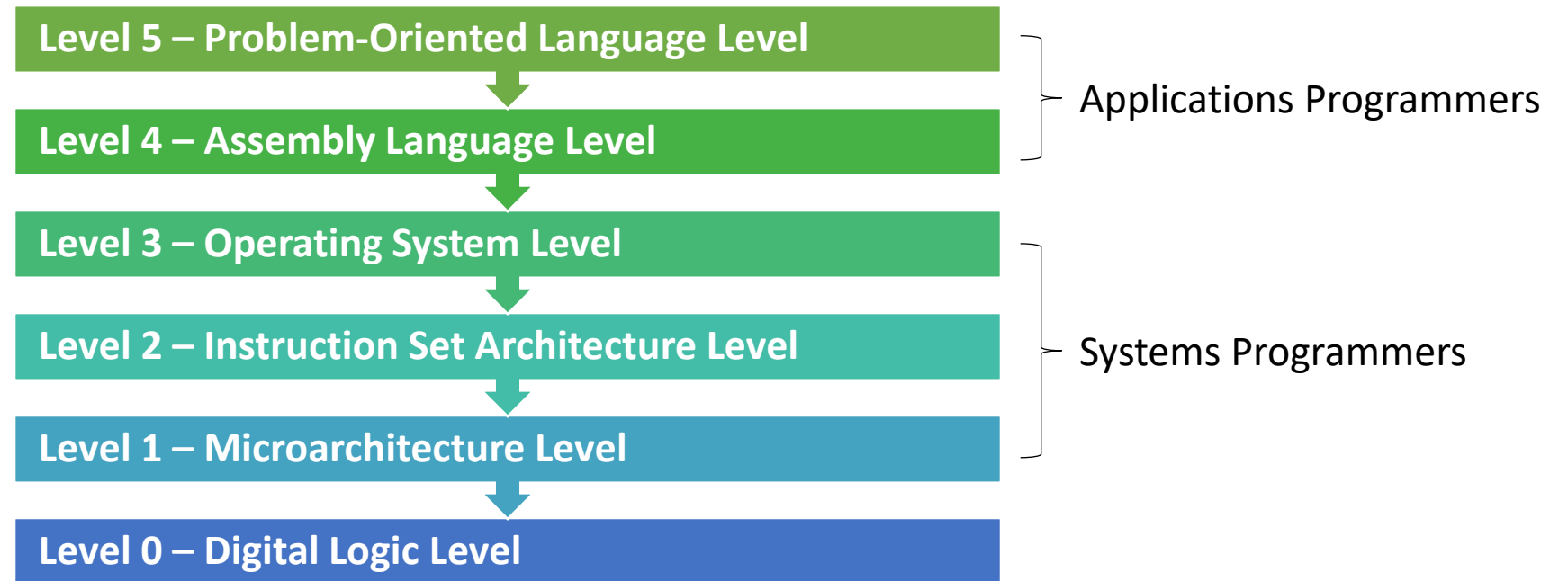
Computer Organization

- High-level programming languages like C and C++.
- A compiler generally translates these programs to the assembly or operating system level.
 - Some languages may be interpreted instead.



Computer Organization

- Application programmers work at higher levels.
- Systems programmers work in the lower levels.



Computer Generations

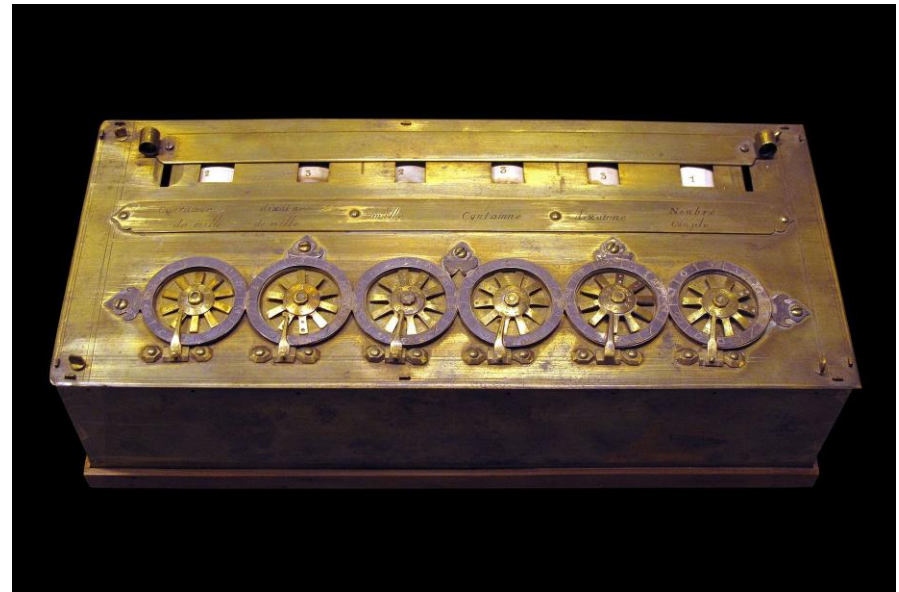
- Zeroth Generation – Mechanical Computers (1640s – 1940s)
- First Generation – Vacuum Tubes (1940s – 1950s)
- Second Generation – Transistors (1950s – 1960s)
- Third Generation – Integrated Circuits (1960s – 1980s)
- Fourth Generation – Very Large-Scale Integration (1980s – Present)

Mechanical Computers

- No electricity.*
 - Operated by hand.
 - *Some used electromechanical parts near the end of the generation.
- Typically made of wood and metal.
- More sophisticated than other tools like an abacus or slide rule.

The Pascaline

- Also known as Pascal's Calculator
 - Invented by Blaise Pascal
- Started work in 1642
 - Finished in 1645
- Only made about two dozen.

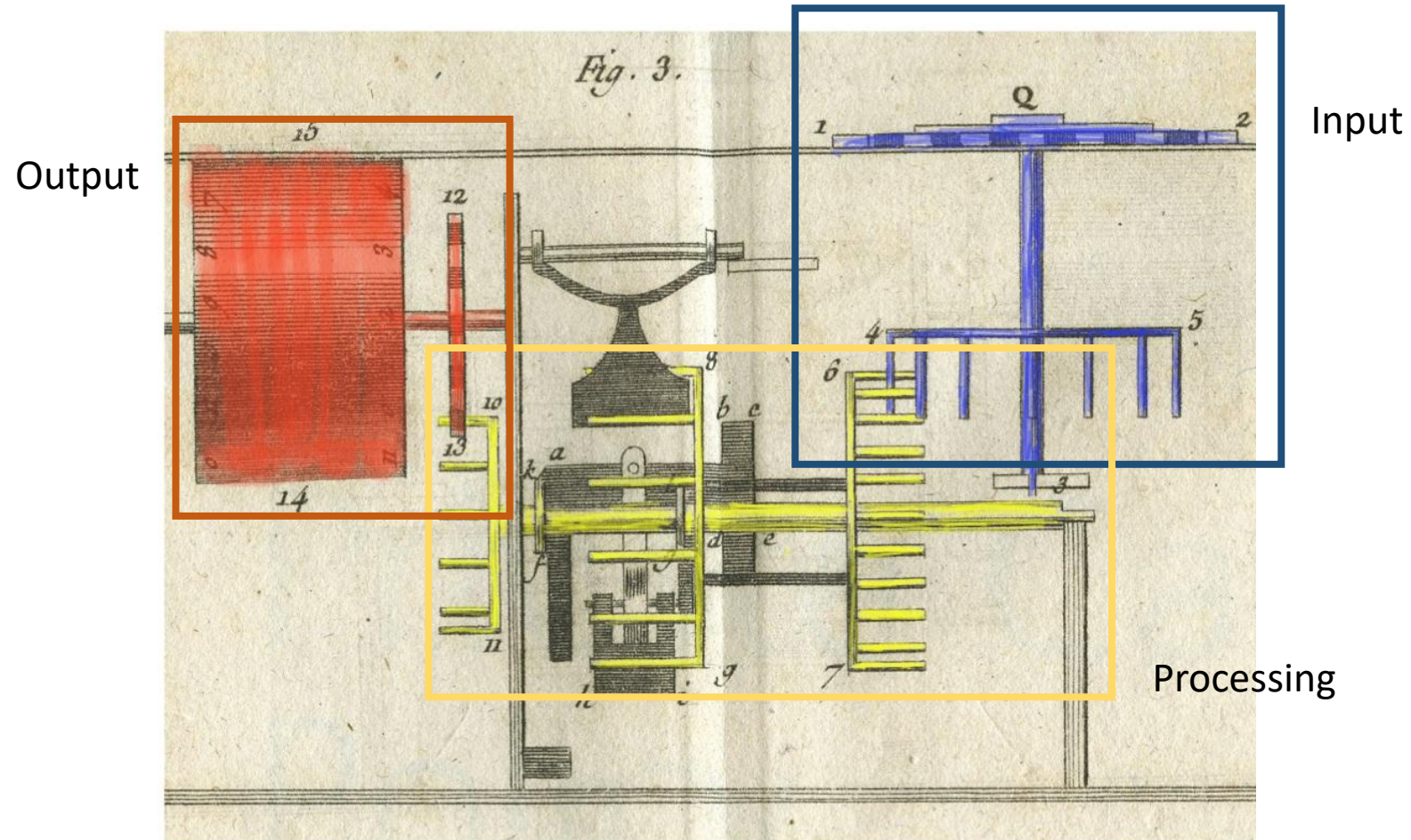


The Pascaline

- Could add and subtract.
- Could multiply and divide (using repeated addition or subtraction).
- Had a few variations:
 - Scientific
 - Accounting
 - Land Surveying

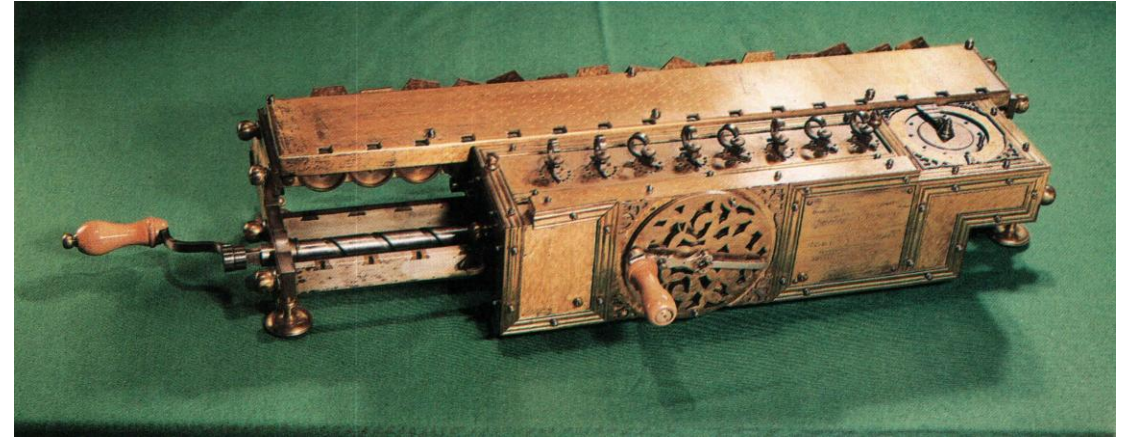
[Pascaline Simulation Video](#)

The Pascaline



Stepped Reckoner

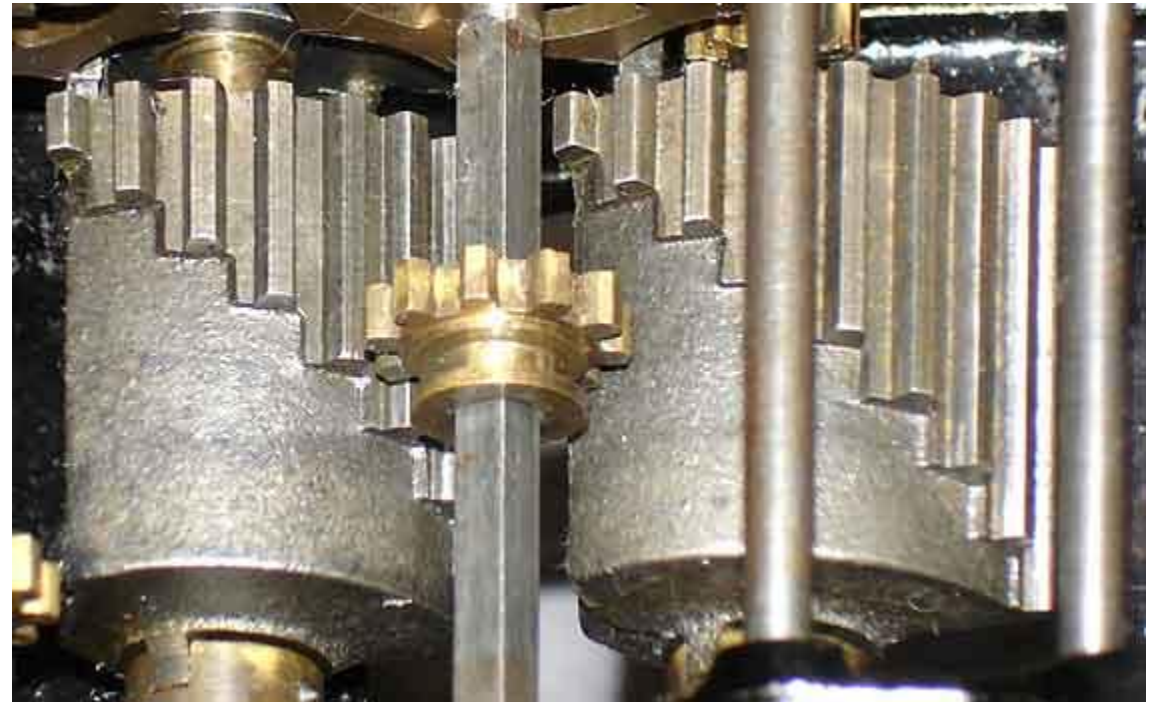
- Invented by Gottfried Leibniz
- Completed in 1694
- Heavily influenced by the Pascaline.
 - Wanted to design a device that could add, subtract, multiply, and divide.
- Required very precise components
 - Reliability problems



Leibniz Wheel/Stepped Drum



[Stepped Drum Simulation Video](#)



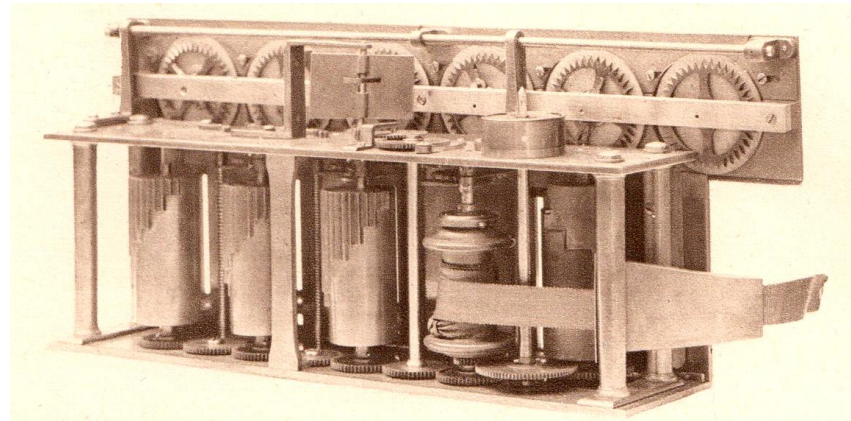
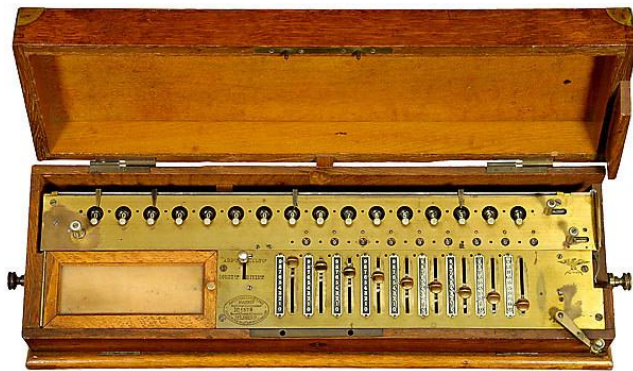
Stepped Reckoner

- Never brought to market.
 - Only two prototypes
 - Showed the potential/possibility of this concept
- The Leibniz Wheel was used in many other mechanical calculators through the 1970s.

Arithmometer

- Created by Charles Thomas
 - French Inventor
- First commercially successful mechanical calculator
 - 1850s to 1890s – only commercially available mechanical calculator.
- Utilized concepts from the Pascaline and Stepped Reckoner.

[Arithmometer Simulation Video](#)



Curta

- Created by Curt Herzstark
 - Austrian Engineer
- First successful portable calculator (1950s/1960s)
 - Up until electronic handheld calculators came out in the 1970s.
- Utilized concepts from the Stepped Reckoner and Arithmometer.

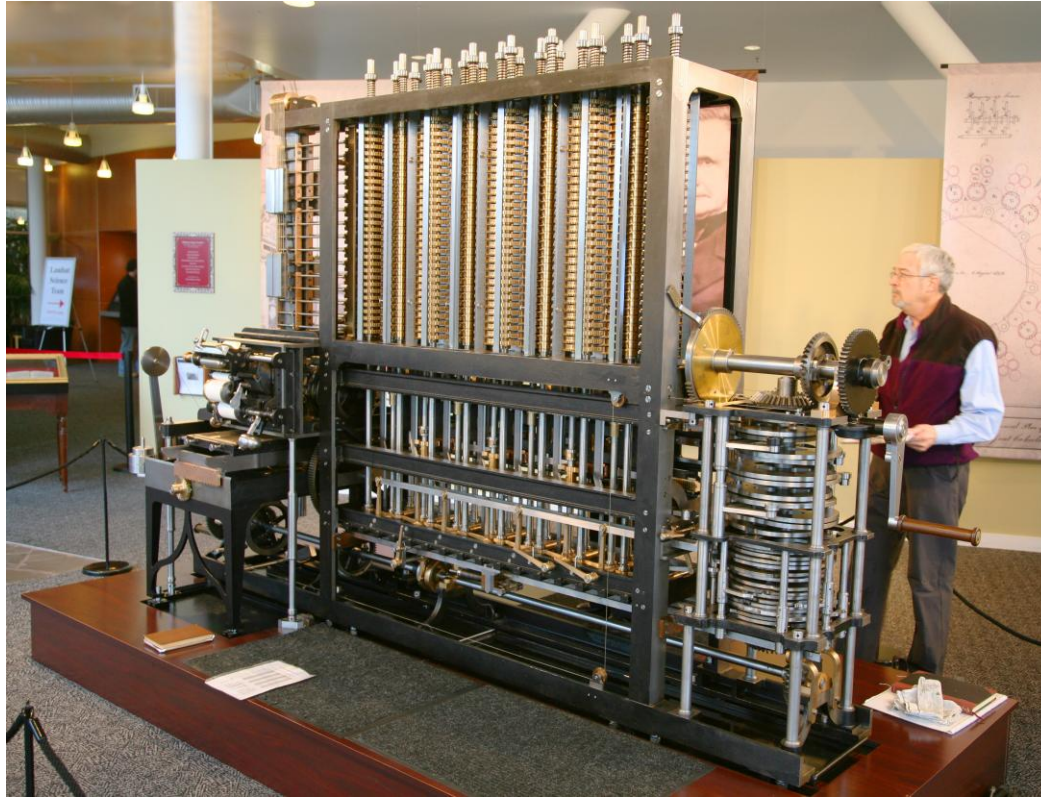


[Curta Simulation Video](#)

Difference Engine

- First theorized in the late 1700s.
- Could tabulate polynomial functions.
- Prototype (Difference Engine No. 0) was completed by Charles Babbage in 1822
- Difference Engine No. 1 was planned and funded but not completed.
- Difference Engine No. 2 was designed but never built.

Difference Engine



[Difference Engine Simulation Video \(Part 1\)](#)

[Difference Engine Simulation Video \(Part 2\)](#)

[Difference Engine Simulation Video \(Part 3\)](#)

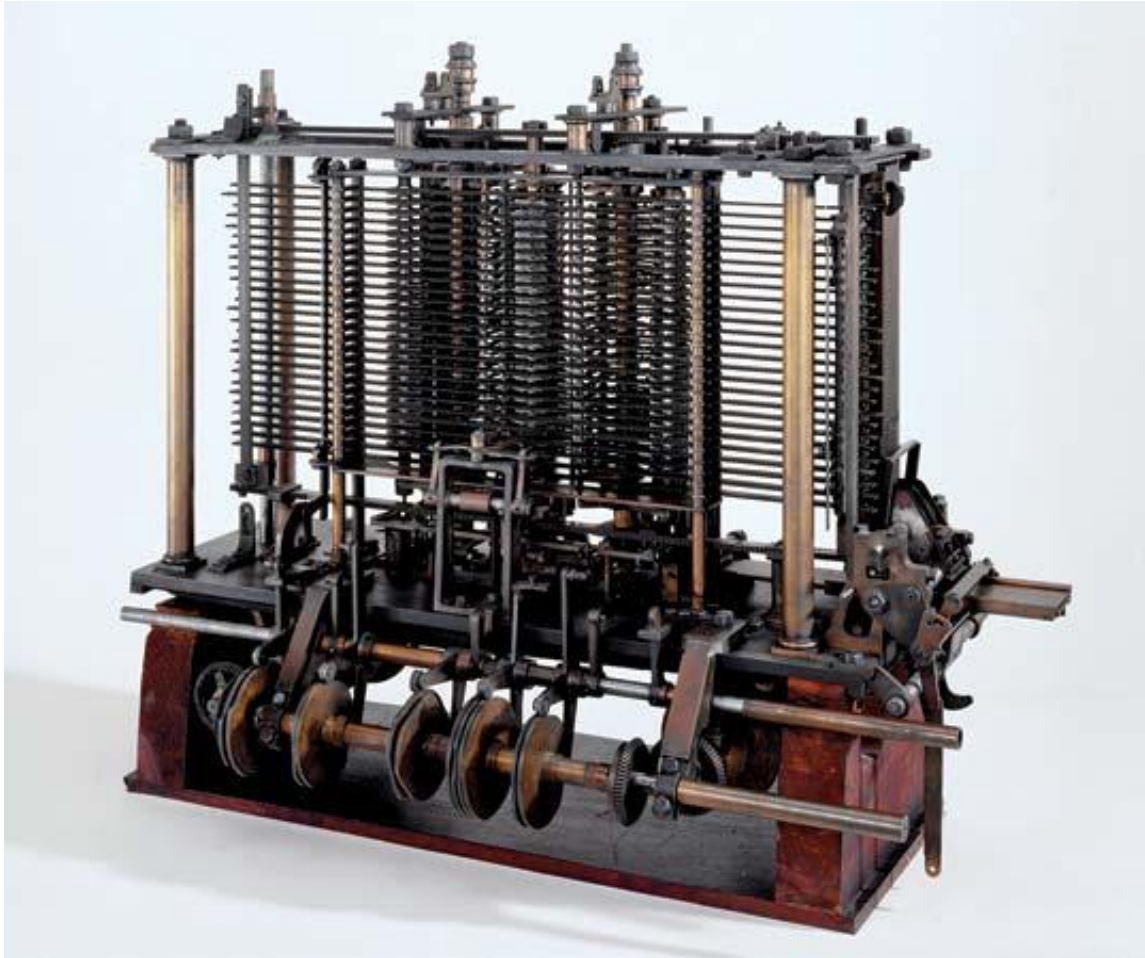
[Difference Engine Simulation Video \(Part 4\)](#)

[Difference Engine Simulation Video \(Part 5\)](#)

Analytical Engine

- Babbage's next project.
 - Started in the 1830s
- Designed to be general-purpose and programmable.
 - Programmed using cards with holes in them (***punched cards***)
- Construction was never completed.
 - It was ahead of its time.
 - The first working general-purpose computers wouldn't come around for another 100 years.

Analytical Engine



[Analytical Engine Simulation Video](#)

Harvard Mark I

- Designed by Howard Aiken/IBM
 - 1930s – 1940s
- Automatic, general-purpose and programmable.
- Among the first to use electromechanical parts
- Computed and printed mathematical tables.
 - Influenced by the Analytical Engine.

Harvard Mark I



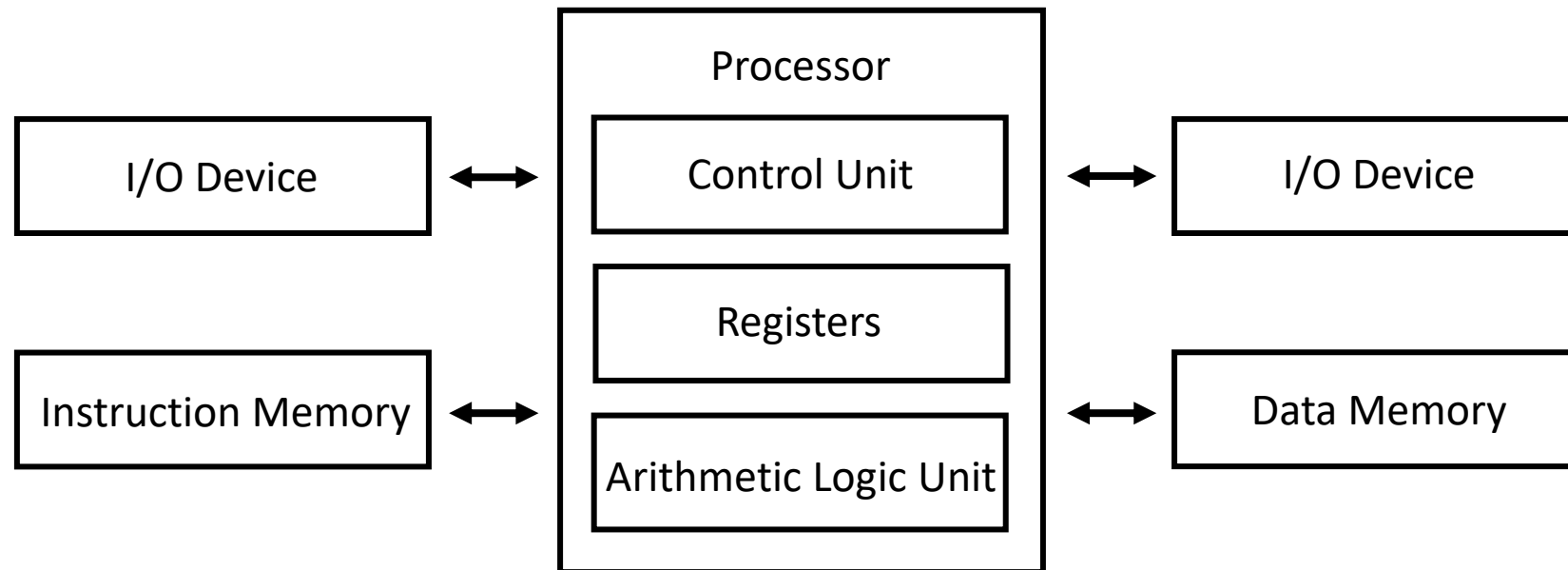
[Harvard Mark 1 Video](#)

[Harvard Mark 1 Video 2](#)

Harvard Architecture

- The Harvard Architecture is the name given to the design first implemented in the Mark I.
- Its configuration enables simultaneous access to instructions and data
 - *Parallelism*
 - Drawback is that it requires duplicating address, data, and control lines to access both memory regions
- Rarely used in modern computer systems

Harvard Architecture

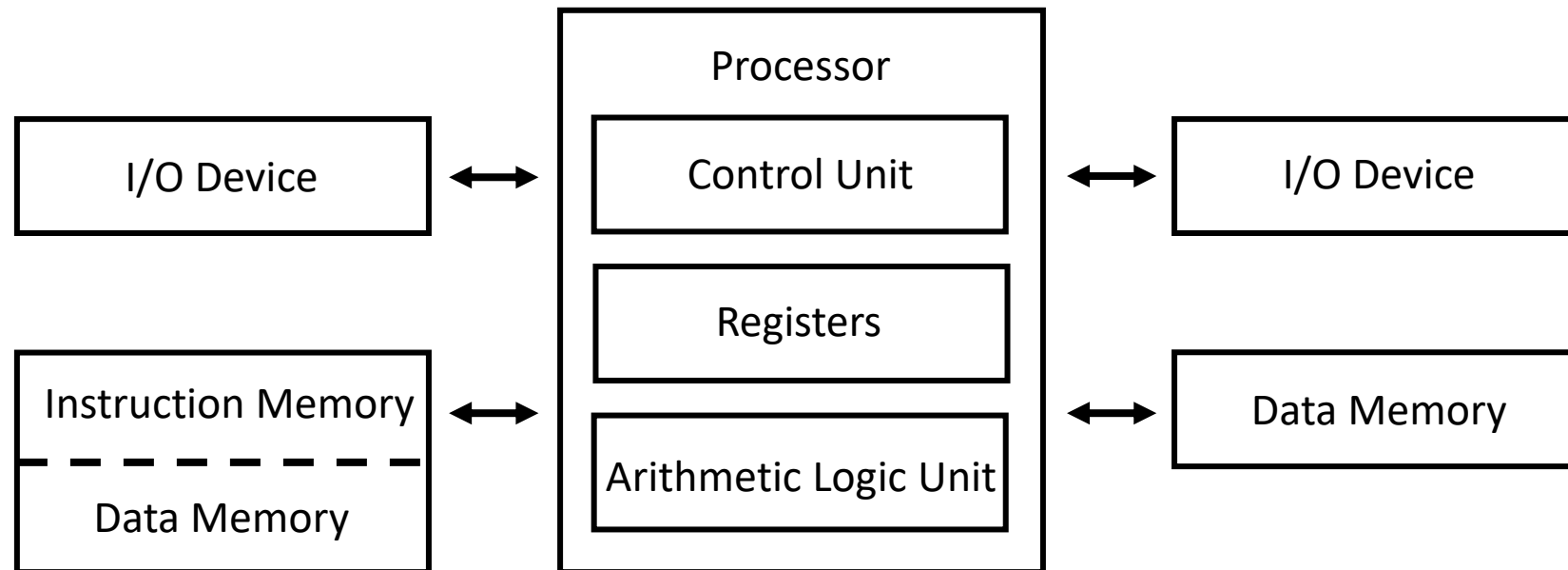


Harvard Architecture

Harvard Architecture

- The modified Harvard Architecture separates program instructions from data memory
- Its configuration enables separate program instruction and data memory regions, and it often supports storing data in program memory and storing instructions in data memory.

Harvard Architecture



Modified Harvard Architecture

Vacuum Tubes

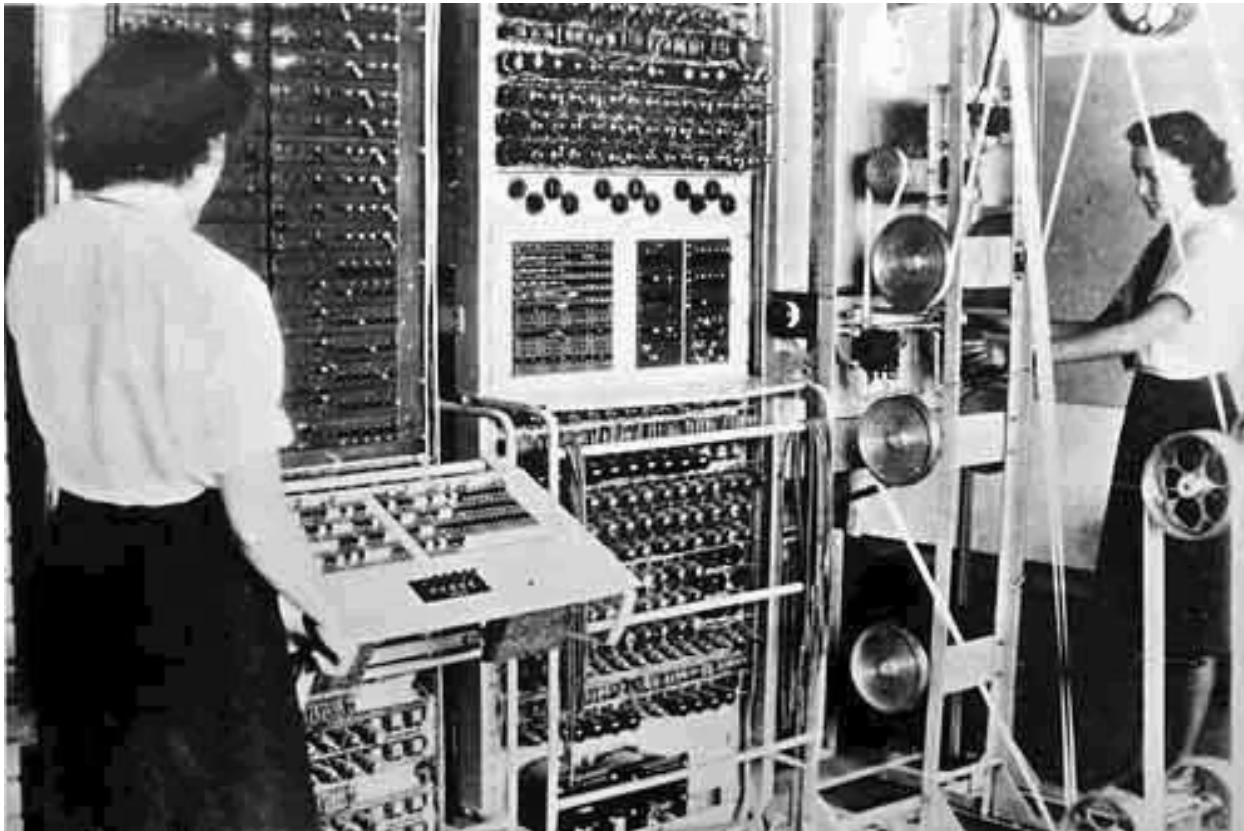
- Replaced the slower electromagnetic relays in early electronic computers.
 - Also used in radios and televisions.
- Controlled electrical flow, like a switch.
- There is a good description of how vacuum tubes work in the transistors video shown later in the lecture.



Colossus

- Designed by Tommy Flowers at Bletchley Park in the 1940s
- Used for codebreaking, specifically the Lorenz cipher, during World War II.
- First electronic, digital computer.
 - Did not store programs in memory.
 - Was not general-purpose.
- Was not public knowledge until the 1970s.

Colossus



[Colossus Video](#)

ENIAC

- Electronic Numerical Integrator And Computer
- Created by John Mauchly and John Presper Eckert (1940s)
 - Built at the University of Pennsylvania
 - February 15th was declared “ENIAC Day” in Philadelphia in 2011.
- Electronic, digital, general-purpose, programmable with stored programs.
 - Wasn’t a secret or undiscovered like Colossus or other electronic computers at that time.

ENIAC

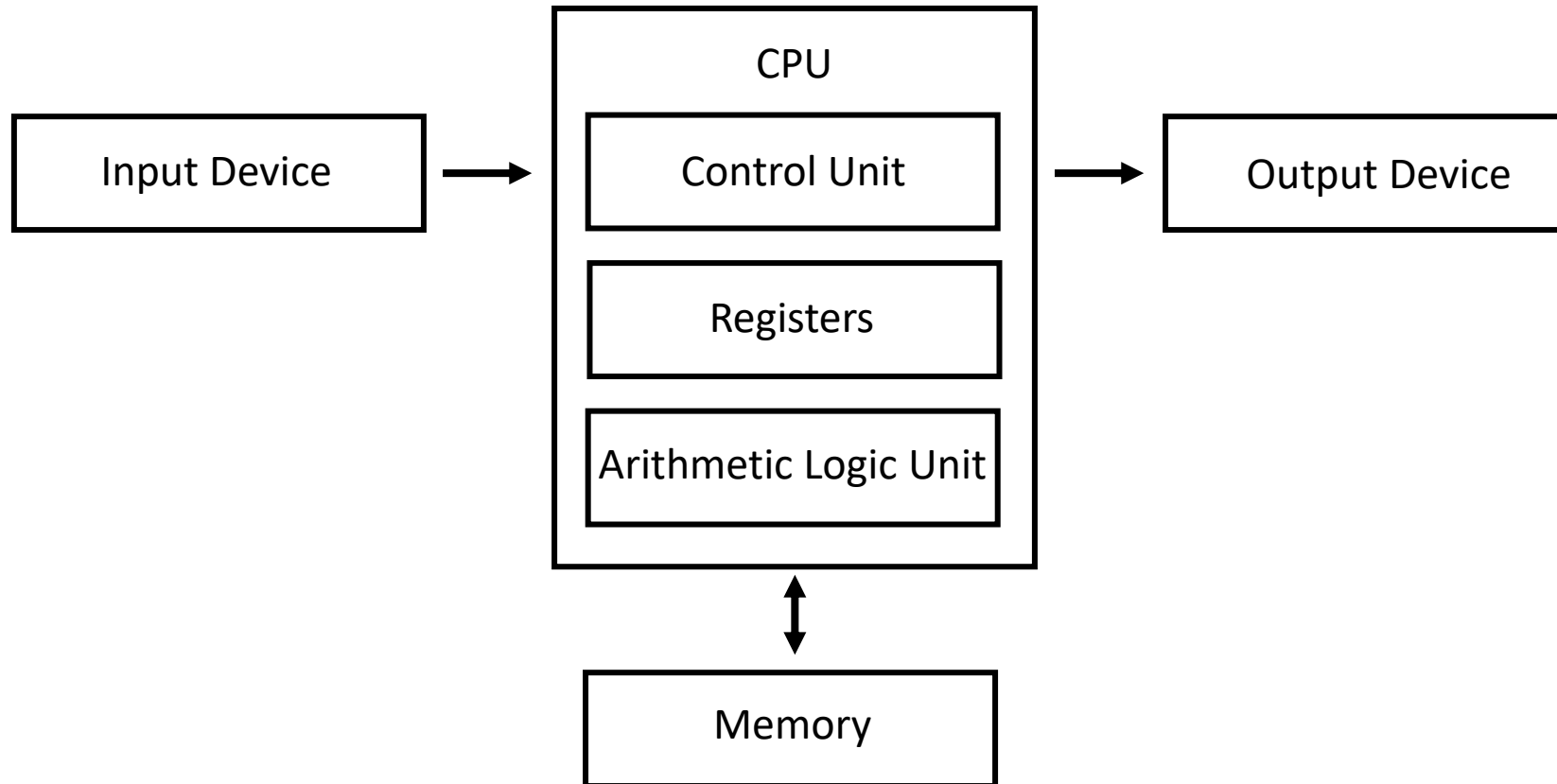


[ENIAC Video](#)

von Neumann Architecture

- The von Neumann Architecture is the computer architecture that modern processors are based on.
 - Named for a computer architecture originally designed by mathematician John von Neumann in 1945
- The von Neumann architecture consists of:
 - The Central Processing Unit (CPU)
 - Arithmetic Logic Unit (ALU)
 - Register file
 - Control unit
 - Memory for storing data and instructions
 - External storage
 - Input and output devices
- Distinguishing feature from Harvard Architectures: a single area of memory

von Neumann Architecture



von Neumann Architecture

von Neumann Architecture

- The von Neumann Architecture is not without some significant issues.
- The most well known is the **von Neumann bottleneck**:
 - Having a single interface between the processor and memory often needs multiple instruction cycles to retrieve a single instruction and the data the instruction requires.
 - Many programs spend most of their time reading and writing to memory.
 - This bottleneck hinders the processor's performance by spending a lot of time accessing the single area of memory

UNIVAC

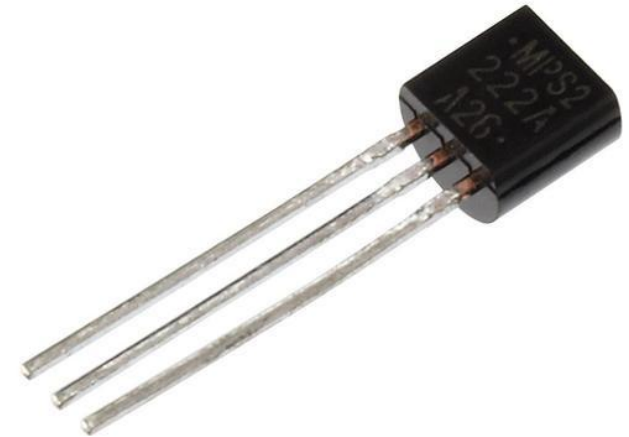
- A family of Universal Automatic Computers
- Created by Mauchly and Eckert (also in the 1940s)
 - Formed their own company
 - Later bought by Remington-Rand.
- First commercially successful electronic, digital, general-purpose, and programmable computers.
 - Originally intended for the Census Bureau.
 - Produced through the 1970s
- Used magnetic tape for memory.
 - Could read punched cards to tape.

UNIVAC



Transistors

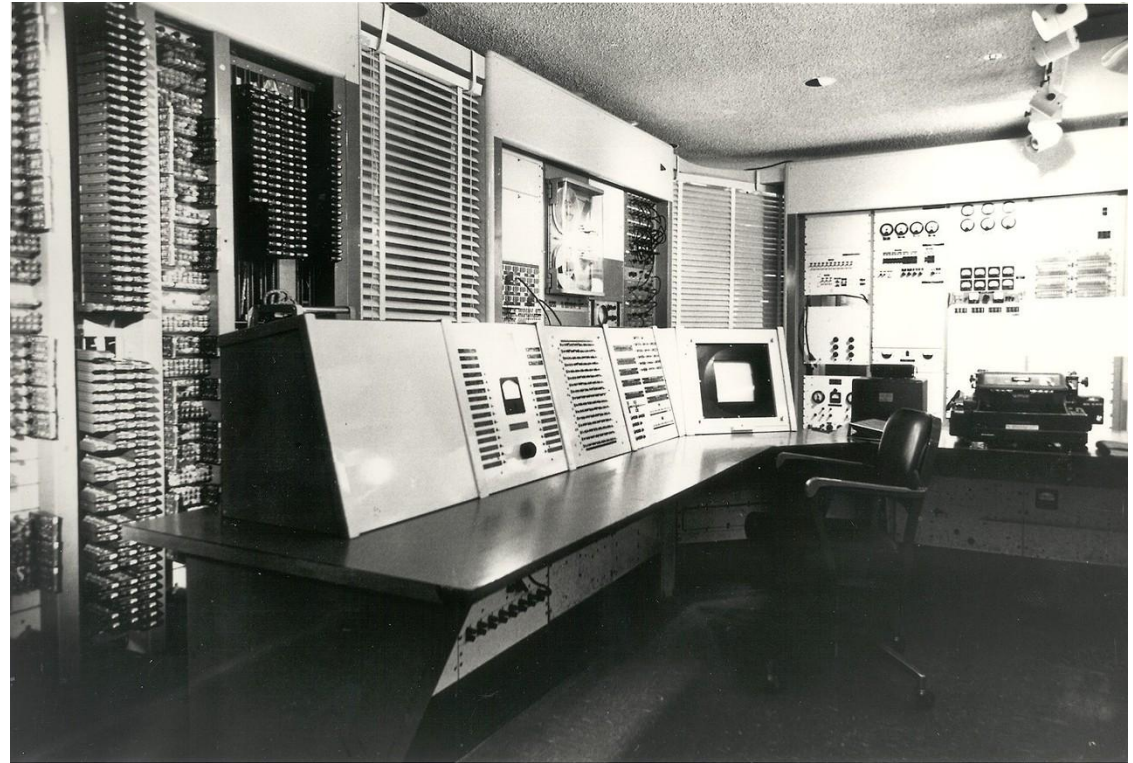
- Replaced vacuum tubes.
- Can control the amount of current or voltage, amplification/modulation or switching of an electronic signal.
- The transistor became the primary building block of computing and electronic equipment as we know it today.
- Magnetic System Memory



[Transistors Video](#)

TX-0

- Fully transistorized computer.
 - Created at MIT (1956)
- Experimental



DEC and the PDP

- Digital Equipment Corporation (1950s – 1990s)
- Created a transistorized line of computers, starting in 1961.
- Programmed Data Processor (PDP)
 - Commercially available

PDP-1

- First of DEC's PDP series.
- A lot of software firsts
 - First computer games
 - First text editor
 - First word processor
 - First debugger
 - First computerized music
- Focused on the user

[PDP-1 Video](#)



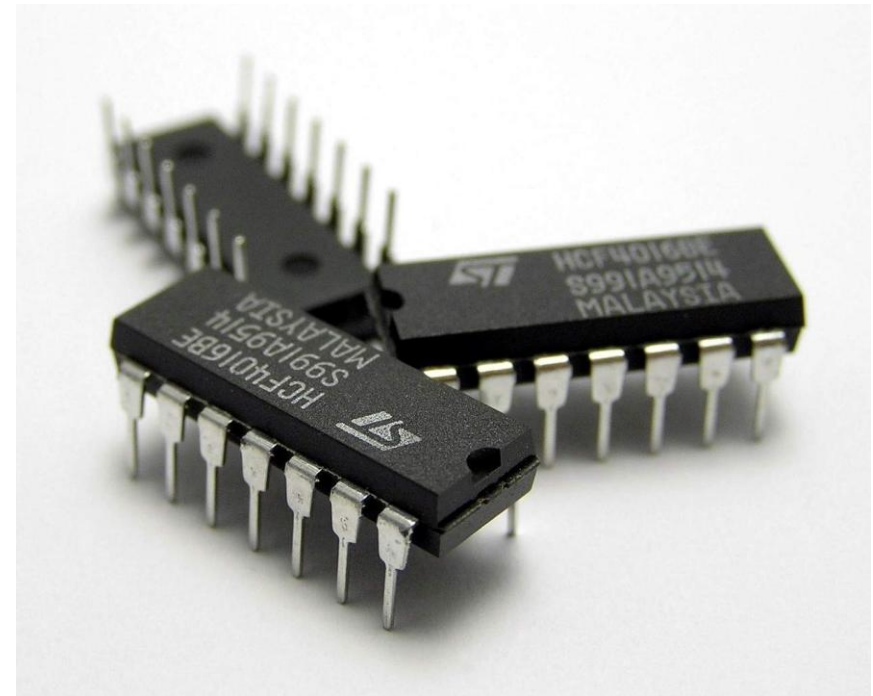
PDP-8

- First commercially successful minicomputer.
 - 1965
- Affordable and accessible to new businesses.
- Focused on the value.



Integrated Circuits

- Holds a large number of transistors on a single “chip”.



[Integrated Circuits Video](#)

IBM 360

- Designed for commercial and scientific applications.
 - 1960s – 1970s
- Scalable
 - Add or remove components
- Software written for one could be run on others.
- Several programs in memory at once.



[IBM 360 Video](#)

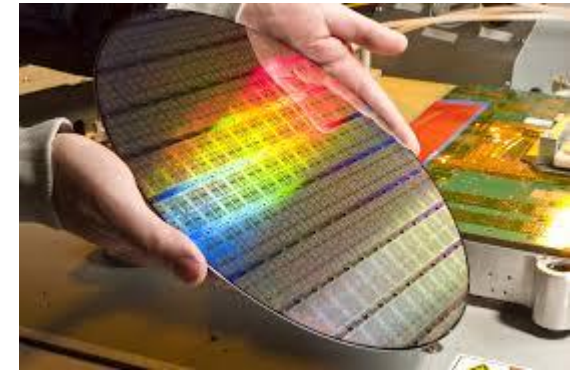
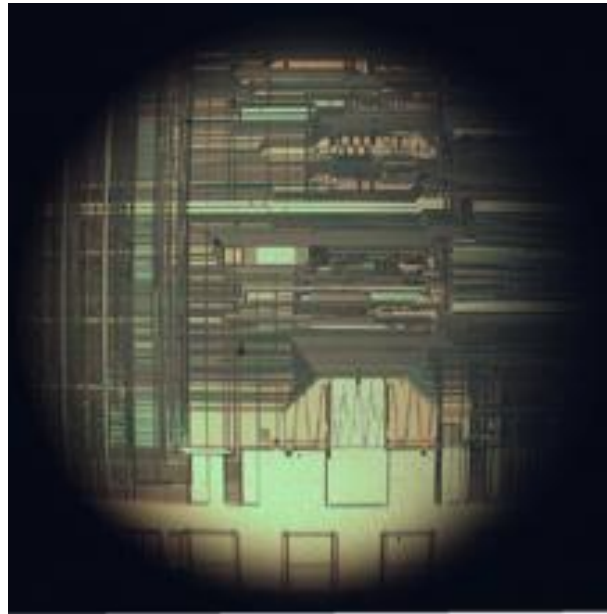
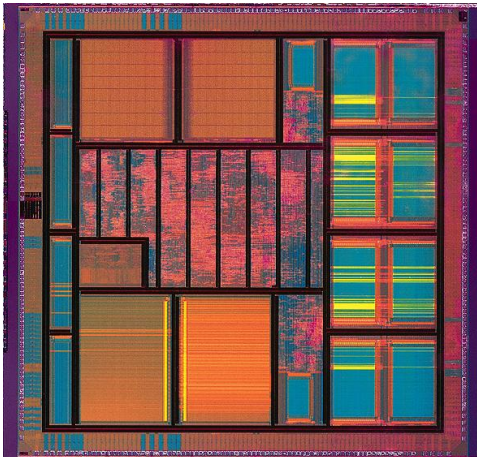
PDP-11

- DEC's most successful minicomputer.
 - 1970s to 1990s
- Large number of peripheral devices.
- Could be upgraded/scaled as needed.



Microprocessors and VLSI

- Billions of nanoscale transistors and multiple integrated circuits in one device.
 - This process is mentioned at the end of the Integrated Circuits Video.



Personal Computers

